이미지 처리 인프라 확장 전략 분석

Lambda 기반 처리에서 Auto Scaling 컴퓨팅(ECS Fargate)으로 단계적 전환

클라우드 컴퓨팅
202312710 신선하

프로젝트 출발점:

자동 사진 정리 및 위치 기반 앨범 서비스

전체 서비스 아키텍처 (초기안)



완전 서버리스

서버 프로비저닝 및 관리 불필요

이벤트 기반

S3 업로드 이벤트로 자동 실행

관리 포인트 최소화

인프라 설정 시간 단축 및 운영 용이

이미지 분석 & 메타데이터 처리

ESIF 데이터 추출

- 촬영 날짜 및 시간
- GPS 좌표 (위도, 경도)
- 카메라 모델 및 제조사 정보

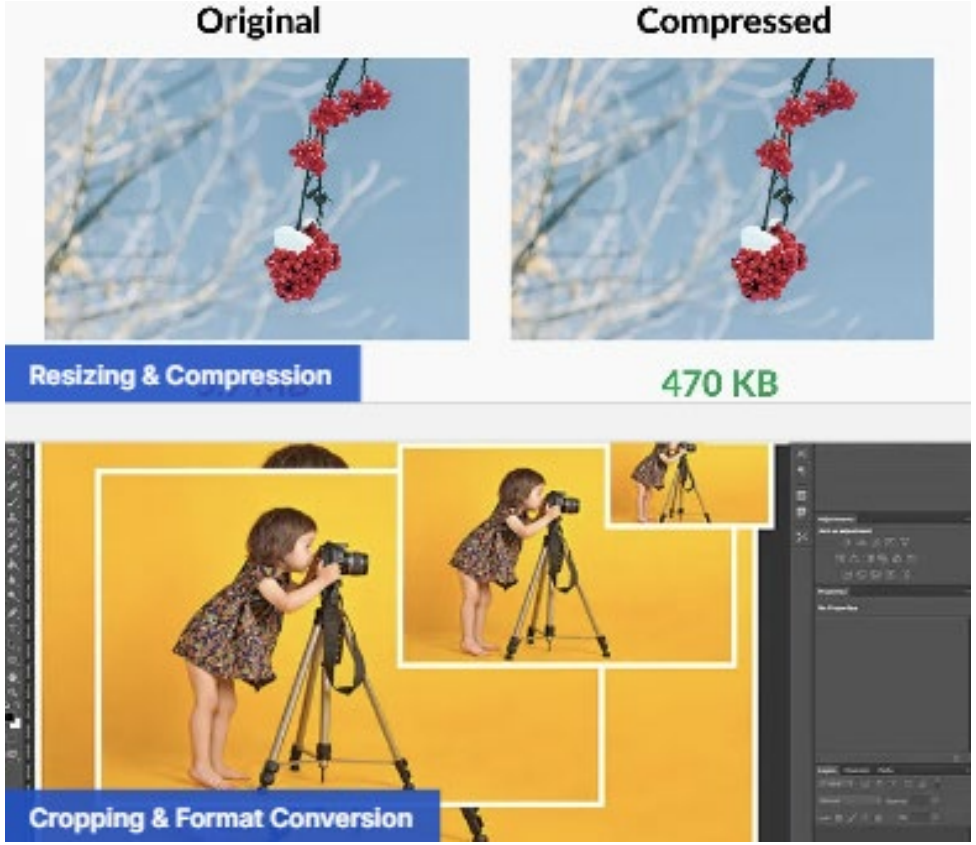
이미지 품질 분석

- 해상도
- 파일 크기 및 압축률
- 이미지 포맷

메타데이터 저장

- DynamoDB에 구조화된 JSON 저장
- 위치/시간 기반 빠른 검색을 위한 인덱싱

그런데 이미지 처리 작업은?



CPU Intensive

Lambda 환경에서 픽셀 단위의 복잡한 연산 필요
(리사이즈, 필터)

Memory Intensive

Lambda 메모리 제한 환경에서 사용량 급증

실험 목적

01 Lambda 처리 가능성 검증
서버리스(Lambda) 환경에서 실제 이미지 처리 작업이 기술적으로 가능한지 검증

02 성능 지표 측정
이미지 크기별 처리 시간과 메모리 사용량 데이터 수집

03 임계점 파악
어떤 조건(파일 크기, 동시성)에서 Lambda 타임아웃이나 에러가 발생하는지 식별

04 확장 필요성 근거 마련
추측이 아닌 정량적 데이터를 통해 인프라 확장의 당위성 보장

Lambda-only 아키텍처



S3 Bucket

이미 업로드 시 자동으로 이벤트 알림 생성

Amazon SQS

이벤트 큐에 저장하여 비동기 처리 및 재시도 보장

AWS Lambda

SQS를 폴링하여 이미지 처리 로직 수행

Lambda-only 아키텍처

Test Data (Images)

Small	1~2 MB (JPEG)
Mdeium	5~8 MB (JPEG)
Large	8~10 MB (JPEG)

Compute (Lambda)

Runtime	Python 3.10
Memory Config	128 MB / 512 MB / 1024 MB
Timeout	3 sec (Baseline)

Infrastructure

Region	Ap-northeast-2 (Seoul)
Library	Pillow (Lamda Layer)

Load Conditions

Trigger	S3 Object Upload Events → SQS → Lambda
Invocation Pattern	Sequential /Burst Uploads

Lambda 이미지 처리 로직

01 이미지 로드 (Load)
S3 버킷에서 원본 이미지를 스트림 형태로 다운로드

02 이미지 처리 (Process)
Pillow 라이브러리를 사용하여 리사이즈 및 압축 수행

03 버퍼 저장 (Buffer)
처리된 이미지를 메모리 버퍼(BytesIO)에 저장

04 업로드 (Upload)
최적화된 이미지를 대상 S3 버킷에 업로드

```
obj = s3.get_object(Bucket=bucket, Key=key)
image = Image.open(obj["Body"]).convert("RGB")
```

```
image.thumbnail((1024, 1024))
```

```
buf = io.BytesIO()
image.save(buf, format="JPEG", quality=85)
buf.seek(0)

out_key = OUTPUT_PREFIX + key.split("/")[-1]
```

```
s3.put_object(
    Bucket=bucket,
    Key=out_key,
    Body=buf,
    ContentType="image/jpeg"
)
```

실행 로그 & 실제 처리 화면

```
2025-12-21T19:05:38.976+09:00	INIT_START Runtime Version: python:3.10.v109 Runtime Version ARN: arn:aws:lambda:ap-northeast-2::runtime:c3d97831572c48429cac556b7614e4000136170a0ddcdf1dca3854ba043bc573
2025-12-21T19:05:39.612+09:00	START RequestId: 3d1f3db1-c944-5d63-9d29-5c7001ed644b Version: $LATEST
2025-12-21T19:05:39.613+09:00	Lambda image processor started (S3→SQS→Lambda)
2025-12-21T19:05:39.613+09:00	{"Records": [{"messageId": "3a8fef88-4fc9-4b1d-8981-885d606b4f0d", "receiptHandle": "AQEBTOw7LUSLmQK1XL0X3NpyeiobhOW98b7Chc6FwFx9ZQtvLqNvegtaRhn4Q3TJ1ubv7mBt4jkajaERXDG1X
2025-12-21T19:05:39.613+09:00	Processing photo-pipeline-raw/input-lambda/ocean2.jpg
2025-12-21T19:05:41.427+09:00	Uploaded output-lambda/ocean2.jpg
2025-12-21T19:05:41.428+09:00	END RequestId: 3d1f3db1-c944-5d63-9d29-5c7001ed644b
2025-12-21T19:05:41.429+09:00	REPORT RequestId: 3d1f3db1-c944-5d63-9d29-5c7001ed644b Duration: 1815.84 ms Billed Duration: 2449 ms Memory Size: 512 MB Max Memory Used: 178 MB Init Duration: 632.26 ms
2025-12-21T19:07:39.650+09:00	START RequestId: 6760e102-35ce-58f0-a55c-80c3fcd6f4a5 Version: $LATEST
2025-12-21T19:07:39.654+09:00	Lambda image processor started (S3→SQS→Lambda)
2025-12-21T19:07:39.654+09:00	{"Records": [{"messageId": "d497f86a-7564-49a5-8191-c6557f89b0b0", "receiptHandle": "AQEBrwZ6yGxXwXwLPEYtbF3wfenokds90ZOln7LDp+feCR7hFqonobXyFR8puVt+Xeom/B2gIBOi7UJWM8MoL
2025-12-21T19:07:39.654+09:00	Processing photo-pipeline-raw/input-lambda/ocean7.jpg
2025-12-21T19:07:41.568+09:00	Uploaded output-lambda/ocean7.jpg
2025-12-21T19:07:41.571+09:00	END RequestId: 6760e102-35ce-58f0-a55c-80c3fcd6f4a5
2025-12-21T19:07:41.571+09:00	REPORT RequestId: 6760e102-35ce-58f0-a55c-80c3fcd6f4a5 Duration: 1920.49 ms Billed Duration: 1921 ms Memory Size: 512 MB Max Memory Used: 271 MB
```

이미지 크기

2.6 MB
13.3 MB

처리 시간

1816 ms
1920 ms (약 +100ms)

메모리 사용

178 MB
271 MB (+93MB)

과금 시간

2449 ms
1921 ms

Lambda-only 실험 결과 요약

Execution Time

이미지 크기 증기 (x6)에 비해
실행 시간 증가는 제한적 (6%)

리사이즈 연산의 고정 비용 영향이 큼

Memory Usage

178MB vs 271MB → 52% 증가

고해상도 이미지 로딩 시 메모리 사용량
민감하게 증가

단순 Lambda 구조
이미지 크기 증가시
비용 효율성 & 안정성 구조적 한계

Lambda-only 구조 한계

처리 시간 불안정

요청 단위로 실행 환경이 생성되는 구조
Clod Start 및 실행 시간 편차 발생

→ 대용량 이미지 처리 시 성능 예측 어려움

메모리 임계점 도달

함수 단위 메모리 상한 구조 보유
고해상도 이미지 처리시 메모리 증가

→ OOM 위험 + 비용 증가로 직결

비용 효율성 저하

CPU 성능이 메모리에 종속
CPU 집약적 이미지 처리 작업

→ 성능 개선이 곧 비용 증가로 연결

동시성 병목

트래픽 급증 시 리전/계정 단위 Concurrency Limit
도달하여 요청 거부 발생

→ 대규모 파이프라인 확장 시 병목

단순 Lambda 구조는 대규모/고해상도 이미지 처리 파이프라인에 부적합

확장 전략 아이디어

Lambda + ECS



Lambda (Event)

트리거, 가벼운 작업, 불규칙한 트래픽 처리

ECS (Worker)

무거운 이미지 처리, 긴 실행 시간, 안정적 처리

아키텍처 구성 요소

사용된 AWS 서비스



Amazon S3

이미지 저장 및 이벤트 트리거



Amazon CloudWatch

모니터링, 로깅, 성능 분석



Amazon SQS

작업 분리를 위한 메시지 큐 (버퍼 역할)



Amazon Lambda

비교대상 1: **함수** 기반 서버리스 컴퓨팅 (이벤트 처리 / 제어 역할)



Amazon EFS (Fargate)

비교대상 2: **컨테이너** 기반 서버리스 컴퓨팅 (이미지 처리 워커)

Amazon ECS

Elastic Container Service – 완전 관리형 컨테이너 오케스트레이션

Docker Container

애플리케이션과 종속성을 패키징하여
환경에 관계없이 동일한 실행 보장

No Time Limits

Lambda 15분 제한과 달리
장시간 실행되는 이미지 처리 작업에 적합

AWS Fargate

서버 관리 없이 컨테이너를 실행하는
서버리스 컨테이너 실행 방식



역할 분리 설계

The Dispatcher
AWS Lambda

✓ **Event Trigger**
S3 업로드 이벤트 즉시 감지

✓ **Validation**
파일 형식 및 메타데이터 유효성 검사

✓ **Queueing**
작업 정보 SQS 메시지로 변환하여 전송

The Worker
Amazon ECS

✓ **Heavy Processing**
고해상도 이미지 리사이징 및 압축

✓ **Long-running**
시간 제한 없는 안정적인 작업 수행

✓ **Batch-Job**
대량의 이미지를 병렬로 효율적 처리

가벼운 작업은 Lambda가, 무거운 작업은 ECS가 전담하여 효율성 극대화

Lambda 이미지 처리 로직

01 이미지 로드 (Load)
S3 버킷에서 원본 이미지를 스트림 형태로 다운로드

02 이미지 처리 (Process)
Pillow 라이브러리를 사용하여 리사이즈 및 압축 수행

03 버퍼 저장 (Buffer)
처리된 이미지를 메모리 버퍼(BytesIO)에 저장

04 업로드 (Upload)
최적화된 이미지를 대상 S3 버킷에 업로드

```
obj = s3.get_object(Bucket=bucket, Key=key)
image = Image.open(obj["Body"]).convert("RGB")
```

```
image.thumbnail((1024, 1024))
```

```
buf = io.BytesIO()
image.save(buf, format="JPEG", quality=85)
buf.seek(0)

out_key = OUTPUT_PREFIX + key.split("/")[-1]
```

```
s3.put_object(
    Bucket=bucket,
    Key=out_key,
    Body=buf,
    ContentType="image/jpeg"
)
```

S3 이벤트 알림 메커니즘

파일 업로드부터 SQS 메시지 발행까지의 흐름

Dockerfile

파일 편집 보기 H1

```
FROM python:3.10-slim

WORKDIR /app

RUN pip install boto3 pillow

COPY app.py .

CMD ["python", "app.py"]
```

Entrypoint

SQS Polling Loop 실행
메시지 도착 시 이미지 처리 작업 트리거

Application Logic

S3에 저장된 원본 이미지 불러와 리사이징 및 압축 처리 수행
결과 이미지 다시 S3에 저장

Dependencies

Boto3: S3, SQS 등 AWS 서비스 연동
Pillow: 이미지 로딩, 리사이징, 포맷 변환 처리

Base Image

Python 3.10 Slim 기반 이미지로 불필요한 구성 요소 제거
컨테이너 크기 감소 및 빠른 실행 환경 제공

실제 구현 화면

ECS Task 정의

↓ Environment Variable

```
"containerDefinitions": [
  {
    "cpu": 0,
    "environment": [
      {
        "name": "QUEUE_URL",
        "value": "https://sqs.ap-northeast-2.amazonaws.com/250466514297/photo-queue-ecs"
      }
    ],
    "essential": true,
    "image": "250466514297.dkr.ecr.ap-northeast-2.amazonaws.com/photo-processing-ecs:latest",
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/photo-processing-ecs-task",
        "awslogs-region": "ap-northeast-2",
        "awslogs-stream-prefix": "ecs"
      },
      "secretOptions": []
    `
```

↑ Container Image

↑ Logging

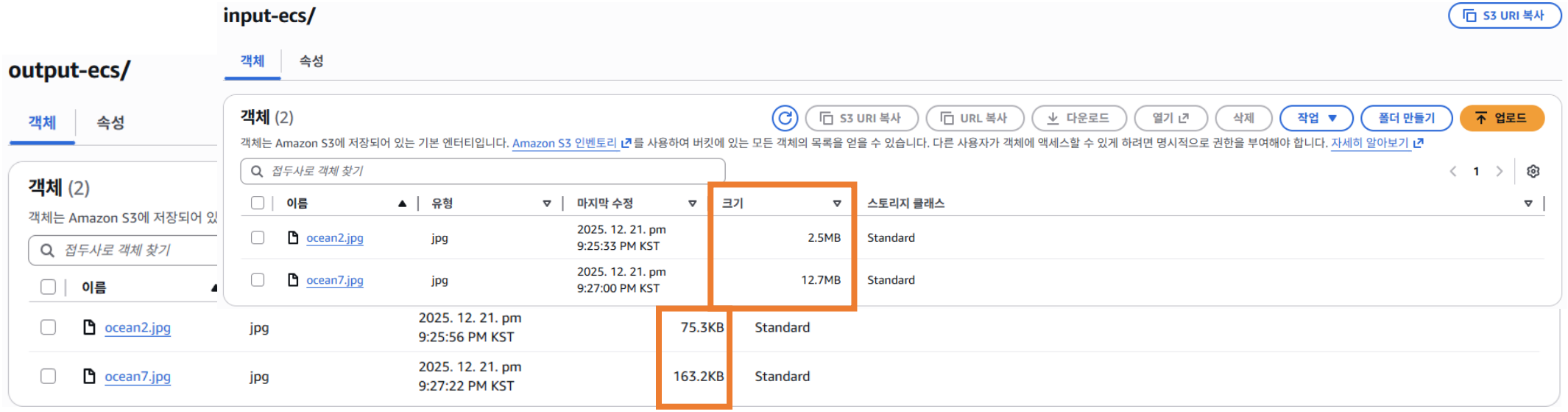
```
    },
    "memory": 512,
    "mountPoints": [],
    "name": "photo-processing",
    "portMappings": [],
    "systemControls": [],
    "volumesFrom": []
  }
],
"cpu": "256",
"executionRoleArn": "arn:aws:iam::250466514297:role/ecsTaskExecutionRole",
"family": "photo-processing-ecs-task",
"memory": "512",
"networkMode": "awsvpc",
"requiresCompatibilities": [
  "FARGATE"
],
```

↓ Task Size

Network Mode

ECS 처리 결과

S3 input / output



이미지 리사이징 및 압축 처리 완료

ECS CloudWatch Logs

```
APP VERSION: 2025-12-21 v2
ECS resize worker started
Processing photo-pipeline-raw/input-ecs/ocean2.jpg
Uploaded output-ecs/ocean2.jpg
```

ECS 컨테이너 워커가 기동

SQS 메시지 수신 후 실제 처리 시작

처리 결과 S3에 업로드

실행 로그 & 실제 처리 화면

```
2025-12-21T21:25:55.404+09:00    APP VERSION: 2025-12-21 v3-metrics
2025-12-21T21:25:55.404+09:00    ECS resize worker started
2025-12-21T21:25:55.404+09:00    Processing photo-pipeline-raw/input-ecs/ocean2.jpg
2025-12-21T21:25:55.404+09:00    Uploaded output-ecs/ocean2.jpg
2025-12-21T21:25:55.404+09:00    METRIC input_bytes=2590100 output_bytes=77129
2025-12-21T21:25:55.404+09:00    METRIC total_ms=1257 download_decode_ms=799 resize_ms=399 encode_ms=3 upload_ms=54
```

```
2025-12-21T21:27:21.459+09:00    APP VERSION: 2025-12-21 v3-metrics
2025-12-21T21:27:21.459+09:00    ECS resize worker started
2025-12-21T21:27:21.459+09:00    Processing photo-pipeline-raw/input-ecs/ocean7.jpg
2025-12-21T21:27:21.459+09:00    Uploaded output-ecs/ocean7.jpg
2025-12-21T21:27:21.459+09:00    METRIC input_bytes=13359630 output_bytes=167139
2025-12-21T21:27:21.459+09:00    METRIC total_ms=1907 download_decode_ms=1406 resize_ms=409 encode_ms=4 upload_ms=85
```

이미지 크기

2.6 MB > 75 KB
13.3 MB > 163 KB

처리 시간

1257 ms
1907 ms (+650ms)

메모리 할당량

512 MB
(Task 단위 고정)

Lambda vs ECS 구조 비교



AWS ECS

vs



AWS Lambda

Execution Model

Lambda: 이벤트 트리거 및 경량 작업 처리

ECS: 이미지 리사이징 등 장시간, 고부하 작업 처리

Resource Limits

Lambda: 실행 시간, 메모리 제한 존재

ECS: Task 단위 리소스 할당, 시간 제한 없음

Best Use Case

Lambda: 이벤트 기반 분기, 검증, 트리거

ECS: 대용량 이미지 처리 및 병렬 워커

Lambda와 EVS로 확장성과 안정성을 동시에 확보하는 구조 구현

성능 관점 비교

	Lambda-only (Small)	ECS (Small)	Lambda-only (Large)	ECS (Large)
처리 시간	~1.8-1.9 s	1.26 s	~1.9-2.0 s	1.91 s
처리 시간 증가	소폭 변동 (~100 ms)	-	-	+650 ms
메모리 특성	178 MB	할당량 512 MB	271 MB	할당량 512 MB
주요 병목	메모리/CPU	I/O	메모리 증가	I/O

소용량 이미지 처리: Lambda-only로 충분
대용량 이미지 처리: ECS 기반 구조가 더 안정적이고 예측 가능

안정성 관점 비교

	Lambda-only	ECS
타임아웃 위험	대용량 처리 시 증가	없음
메모리 초과(OOM)	고해상도 시 가능	할당량 내 안정
처리시간 내 변동성	큼	작음
트래픽 폭주 대응	Concurrency 제한	SQS 큐 기반 완충
실패 시 처리	재시도 어려움	재처리 가능

Lambda-only

소량, 단발성 처리에는 충분
대량, 지속 처리 시 예측성 저하

ECS

처리 시간이 일정하고 예측가능
큐 기반 구조로 과부하 흡수 가능

확장성 관점 비교

AWS Lambda

✓ Fast Scale-out
요청 단위로 즉각적인 확장 가능

✓ Concurrency Limit
동시 실행 수 제한 존재

✓ Throttling Risk
대량, 지속 트래픽 시 요청 지연 또는 거부 가능

Amazon ECS

✓ Queue-driven Scaling
SQS 대기열 길이에 따라 워커 수 확장

✓ Sustained Throughput
지속적인 대향 작업 처리에 적합

✓ Back-pressure Handling
과부하 시 큐에 적재 후 순차 처리

순간적인 이벤트 처리에는 Lambda-only
지속적인 대량 이미지 처리에서는 SQS 기반 ECS 워커 구조

실험으로 증명한 포인트



언제 Lambda가 한계?

Time Constraints

실행 시간이 길어짐, 예측 불가능한
Long-running 작업 필요시

Resource Spikes

고해상도, 대용량 이미지 처리로
메모리 사용량 급격히 증가

Cost at Scale

지속적인 고부하 트래픽으로 실행 시간 증가
> 비용 빠르게 누적



언제 ECS가 유리?

Stability First

시간 제한 없이 안정적 처리 요구
실패율 최소화해야 하는 대량 처리 환경

Fine-grained Control

OS 레벨의 튜닝이나 특정 라이브러리 의존성이 강한 환경 제어

Predictable Scale

대규모 트래픽 처리 + 처리 시간 및 비용을
예측 가능한 범위 내로 유지 필요

단순한 기술 우위가 아닌, 비즈니스 요구사항(비용, 안정성)에 따른 선택이 핵심

최종 하이브리드 아키텍처

1. Ingestion 이미지 입력 단계

API Gateway를 통해 이미지 업로드
S3 Pre-signed URL 사용하여 부하 없이 직접 업로드
업로드 완료 시 S3 이벤트 발생

2. Decision 처리 경로 결정 단계

Lambda가 S3 이벤트 수신
이미지 메타데이터 (크기, 포맷) 분석
실험 결과 기준 처리 방식 결정
- 소용량 > 단시간 - 대용량 > 장시간 처리

3. Execution Routing

처리 실행 단계 (조건 분기)

Small File – Lambda Direct Processing

Large File – SQS Queue 적재 후 ECS 워커 순차 처리

4. Architecture Characteristic

구조적 특징 요약

Lambda: 이벤트 트리거, 경량 작업 담당

ECS: 장시간, 대용량 이미지 처리 담당

작업 특성에 따라 최적의 컴퓨팅 자원 자동 선택하는 하이브리드 이미지 처리 아키텍처 설계